

La solution : une configuration stratégique des modèles

Comprendre la sélection de modèles

Référence : [Documentation sur le modèle Gemini](#)

Référence : [Tarification de Vertex AI](#)

Référence : [Documentation de référence sur le modèle Vertex AI](#)

Règles de base pour la sélection de modèle :

- **Commencez avec Gemini 2.5 Pro** pour le prototypage et la définition des critères de qualité de référence.
 - Raisonnement excellent pour les analyses complexes et les tâches critiques liées à la qualité
 - Fenêtre de contexte de 2 millions de jetons pour les entrées plus volumineuses
 - Idéal pour le développement et l'évaluation initiaux
- **Optimisez avec Gemini 2.5 Flash** lorsque le coût et la rapidité sont prioritaires.
 - Temps de réponse environ deux fois plus rapides
 - Environ 10 fois moins cher que la version Pro
 - Fenêtre de contexte de 1 million de jetons
 - Raisonnement efficace pour la plupart des tâches simples
 - Idéal pour les charges de travail de production à fort volume
- **Effectuez une analyse des déficits** après la transition de Pro à Flash pour vous assurer que Flash satisfait vos exigences de qualité.

Concepts fondamentaux

1. Configuration avec **GenerateContentConfig**

Référence : [Documentation ADK - Configurer la génération LLM](#)

Extrait de la documentation ADK :

“**generate_content_config (facultatif)** : transmet une instance de [google.genai.types.GenerateContentConfig](#) pour contrôler des paramètres de type **temperature** (randomisation), **max_output_tokens** (longueur de la réponse), **top_p**, **top_k**, et les paramètres de sécurité.”

```
from google.genai import types
```

```
agent = LlmAgent(  
    model="gemini-2.5-flash",  
    generate_content_config=types.GenerateContentConfig(  
        temperature=0.2,          # Résultat plus déterministe  
        max_output_tokens=250,    # Limite la longueur de la réponse  
        top_p=0.8,                # Échantillonnage du noyau  
        top_k=10                  # Échantillonnage top-K  
    )  
)
```

2. Température : créativité ou cohérence

Référence : [Documentation ADK - Configurer la génération LLM](#)

La température contrôle le degré de randomisation de les réponses du modèle :

Température basse (0,0-0,3) - déterministe :

```
factual_config = types.GenerateContentConfig(  
    temperature=0.1 # Très cohérent, prévisible  
)
```

À utiliser pour : l'extraction de données, les questions/réponses factuelles, les résultats structurés

Température moyenne (0,4-0,7) - équilibré :

```
balanced_config = types.GenerateContentConfig(  
    temperature=0.7 # Équilibre entre créativité et cohérence  
)
```

À utiliser pour : le service client, l'enseignement, les conversations générales

Température élevée (0,8-1,0) - créativité :

```
creative_config = types.GenerateContentConfig(  
    temperature=0.9 # Réponses plus variées, créatives  
)
```

À utiliser pour : l'écriture créative, le brainstorming, les textes marketing

3. Paramètres de sécurité

Référence : [Documentation ADK - Exemple de paramètres de sécurité](#)

Référence : [Paramètres de sécurité de Gemini](#)

Configurez les seuils de filtrage de contenu pour les modèles Gemini :

```
from google.genai import types

# Sécurité stricte pour les agents en contact avec le public
strict_config = types.GenerateContentConfig(
    safety_settings=[
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
            threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
        ),
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_HARASSMENT,
            threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
        ),
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_HATE_SPEECH,
            threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
        ),
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_SEXUALLY_EXPLICIT,
            threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
        )
    ]
)
```

Seuils de sécurité :

- **BLOCK_NONE** : aucun filtrage (non recommandé en production)
- **BLOCK_ONLY_HIGH** : bloque uniquement le contenu fortement susceptible d'être nuisible
- **BLOCK_MEDIUM_AND_ABOVE** : bloque uniquement le contenu moyennement et fortement susceptible d'être nuisible
- **BLOCK_LOW_AND_ABOVE** : le plus strict, bloque même le contenu peu susceptible d'être nuisible

Remarque : Pour les modèles autres que Gemini (par exemple, Claude, GPT), reportez-vous à la documentation spécifique du fournisseur pour connaître les options de configuration de la sécurité, car l'API et les paramètres diffèrent de ceux des modèles Gemini.

4. Jetons de sortie et échantillonnage

Référence : [Documentation ADK - Configurer la génération LLM](#)

Contrôlez la longueur et la diversité des réponses :

```
comprehensive_config = types.GenerateContentConfig(
    temperature=0.7,
    max_output_tokens=2000, # Autorise des réponses plus longues
    top_p=0.95,             # Prend en compte les 95 % de la masse de probabilité
```

```

        top_k=40                # Prend en compte les 40 jetons les plus probables à
chaque étape
    )

concise_config = types.GenerateContentConfig(
    temperature=0.3,
    max_output_tokens=100,      # Force les réponses courtes
    top_p=0.8,                  # Échantillonnage plus ciblé
    top_k=10                    # Moins de choix de jetons
)

```

Explication des paramètres :

- **max_output_tokens** : longueur maximale de la réponse (la valeur par défaut varie selon le modèle)
- **top_p** : échantillonnage du noyau, prend en compte les jetons couvrant les P % les plus probables.
- **top_k** : n'échantillonne que les K jetons suivants les plus probables

Exemple pratique

Créons deux agents optimisés pour des tâches différentes : l'un pour l'extraction de données factuelles et l'autre pour le brainstorming créatif.

Étape 1 : Créer le projet

```

adk create model_comparison
cd model_comparison

```

Étape 2 : Écrire des agents optimisés

Remplacez le contenu de `agent.py` par le code suivant :

```

"""
Démonstration de configuration du modèle illustrant l'optimisation pour des tâches
créatives et factuelles.
Démonstration de la fonction generate_content_config d'ADK avec différents
paramètres.
"""

from google.adk.agents import LlmAgent
from google.genai import types

# Agent 1 : Optimisé pour l'extraction de données factuelles
# Utilise une faible température pour la cohérence, une sécurité stricte pour la
justesse
factual_agent = LlmAgent(
    model="gemini-2.5-flash", # Flash est suffisant pour l'extraction
    name="data_extractor",

```

```
description="Extracts factual information with high consistency",
instruction="""Tu es un extracteur de données précis.
```

Extrais les faits exactement comme énoncés. À ne pas faire :

- Ajouter des informations qui ne sont pas présentes dans l'entrée
- Faire des hypothèses ou des déductions
- Utiliser un langage créatif

Sois précis, concis, et déterministe."""

```
generate_content_config=types.GenerateContentConfig(
    temperature=0.1, # Très faible pour plus de cohérence
    max_output_tokens=500,
    top_p=0.8,
    top_k=10,
    safety_settings=[
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
            threshold=types.HarmBlockThreshold.BLOCK_LOW_AND_ABOVE
        )
    ]
)
)
```

Agent 2 : Optimisé pour le brainstorming créatif

Utilise une température élevée pour la créativité, modèle Pro pour de meilleures idées

```
creative_agent = LlmAgent(
    model="gemini-2.5-pro", # Pro pour une créativité supérieure
    name="creative_brainstormer",
    description="Génère des idées créatives et explore les différentes possibilités",
    instruction="""Tu es un partenaire de brainstorming actif.
```

Génère des idées innovantes, diverses et imaginatives. N'hésite pas à :

- Sortir des sentiers battus
- Combiner des concepts inattendus
- Explorer des approches non conventionnelles

Sois créatif, varié, et stimulant."""

```
generate_content_config=types.GenerateContentConfig(
    temperature=0.9, # Élevée pour la créativité
    max_output_tokens=2000, # Autorise des idées détaillées
    top_p=0.95,
    top_k=40,
    safety_settings=[
        types.SafetySetting(
            category=types.HarmCategory.HARM_CATEGORY_DANGEROUS_CONTENT,
            threshold=types.HarmBlockThreshold.BLOCK_MEDIUM_AND_ABOVE
        )
    ]
)
)
```

Pour adk web, nous allons utiliser l'agent factuel comme root_agent

Passons à creative_agent pour tester un comportement différent

```
root_agent = factual_agent
```

Étape 3 : Exécuter et comparer

```
adk web
```

Test avec un agent factuel :

Essayez ce prompt :

```
Extrais les informations clés : "L'iPhone 15 Pro a été commercialisé en septembre 2023 et est proposé à partir de 999 $ pour une capacité de 128 Go."
```

Comportement attendu :

- Réponse cohérente et factuelle à chaque fois
- Extrait exactement ce qui est énoncé
- Pas d'interprétation créative
- Même format pour les prompts répétés

Test avec un agent créatif :

Changez `root_agent = creative_agent` dans le code, redémarrez et essayez :

```
Trouve 5 fonctionnalités innovantes pour un smartphone de nouvelle génération.
```

Comportement attendu :

- Idées variées et créatives
- Réponses différentes à chaque fois
- Suggestions imaginatives
- Élaboration riche et détaillée

Points à noter :

- **Impact de la température** : faible (0,1) = cohérence, élevée (0,9) = créativité
- **Choix du modèle** : Pro pour une qualité de référence et une créativité complexe, Flash pour l'optimisation des coûts
- **Limites de jetons** : 500 pour l'agent factuel, 2 000 pour l'agent créatif
- **Niveaux de sécurité** : plus strict pour l'agent factuel, équilibrés pour l'agent créatif

Points à retenir

- **Choix du modèle** : commencez avec Pro pour le prototypage et pour établir des bases de qualité, puis optimisez avec Flash pour réduire les coûts et augmenter la rapidité
- **Température** : 0,0-0,3 (factuel), 0,4-0,7 (équilibré), 0,8-1,0 (créatif)
- **Importation depuis google.genai** : `from google.genai import types`
- **GenerateContentConfig** : configure la température, les jetons, l'échantillonnage et la sécurité

- **Seuils de sécurité** : de BLOCK_LOW_AND_ABOVE (strict) à BLOCK_ONLY_HIGH (souple)
- **Optimisation en fonction de la tâche** : configurez les paramètres en fonction du cas d'utilisation, et non de manière universelle